



TripGenie

Documentation Technique Complète

Générateur de voyages personnalisés piloté par intelligence artificielle

Alexis Laubert

Holberton School — Démonstration des fondamentaux · Titre RNCP 5 — Développeur Web et Web
Mobile

Mai 2026 · Version 1.0

Sommaire

0 · Préambule et résumé exécutif	7
0.1 À propos de ce document	7
0.2 Résumé exécutif	7
0.3 Fiche signalétique du projet	8
0.4 Conventions de lecture	8
1 · Présentation du projet	9
1.1 Le problème	9
1.2 La proposition de valeur	9
1.3 Public cible	9
1.4 Parcours utilisateur type	9
1.5 Périmètre fonctionnel	10
1.6 Ce que TripGenie n'est pas	10
2 · Méthodologie et gestion de projet	11
2.1 Démarche de développement	11
2.2 Organisation du dépôt	11
2.3 Outils et environnement	12
2.4 Conventions de commit	12
3 · Architecture générale	13
3.1 Vue d'ensemble	13
3.2 Le choix d'un service unique (same-origin)	13
3.3 Cycle de vie d'une requête	14
3.4 Séparation des responsabilités	14
4 · Choix technologiques et justifications	15
4.1 TypeScript de bout en bout	15
4.2 Node.js + Express	15
4.3 React + Vite — pourquoi pas Next.js ?	15
4.4 Zustand + React Query — pourquoi pas Redux ?	16
4.5 PostgreSQL + Supabase — pourquoi pas MongoDB ni Firebase ?	16
4.6 Zod — la validation comme contrat	17
4.7 Tailwind CSS	17
4.8 Vitest — pourquoi pas Jest ?	17

4.9 Synthèse des arbitrages	17
5 • Le backend en détail	19
5.1 Point d'entrée et chaîne de middlewares	19
5.2 Les middlewares de sécurité	19
5.3 Limitation de débit (rate limiting)	19
5.4 Gestion centralisée des erreurs	20
5.5 Les routes REST	20
5.6 Sémantique des codes HTTP	21
6 • Le pipeline d'intelligence artificielle	22
6.1 Pipeline orchestré contre agent autonome	22
6.2 Les étapes du pipeline de génération	22
6.3 Promise.allSettled — un choix technique structurant	23
6.4 Repli multi-LLM en cascade	23
6.5 L'assemblage du pack : ingénierie de prompt	24
6.6 La partie agentique : le chat de modification	24
7 • Intégrations de services externes	25
7.1 Philosophie : la dégradation gracieuse	25
7.2 Cartographie des services	25
7.3 Les chaînes de repli en détail	25
7.4 Protection des clés d'API	25
7.5 Timeouts et robustesse	26
8 • L'algorithme de scoring	27
8.1 Pourquoi un scoring déterministe ?	27
8.2 Pondération par mode de voyage	27
8.3 Exemple : le mode « luxe »	27
8.4 Résultat structuré	28
9 • Le modèle de données	29
9.1 Vue d'ensemble	29
9.2 Les tables principales	29
9.3 Choix de conception	30
9.4 Isolation des données par utilisateur	30
9.5 Protection contre l'injection SQL	30
10 • Authentification et sécurité	31

10.1 Authentification par JWT en cookie httpOnly	31
10.2 Pourquoi un cookie httpOnly plutôt que localStorage ?	31
10.3 Hachage des mots de passe	32
10.4 Défense en profondeur — cartographie	32
10.5 Attaques explicitement testées	32
10.6 Limites de sécurité connues	33
11 · Le frontend React	34
11.1 Structure SPA et routage	34
11.2 Les maquettes des écrans	34
11.3 Gestion d'état	34
11.4 Communication avec l'API	34
11.5 Expérience utilisateur	35
11.6 Responsive et accessibilité	35
12 · Conteneurisation Docker	36
12.1 Pourquoi Docker ?	36
12.2 Dockerfile multi-étapes	36
12.3 Orchestration locale (docker-compose)	36
13 · Déploiement en production	38
13.1 Hébergement sur Render (PaaS)	38
13.2 Configuration déclarative (render.yaml)	38
13.3 Build et démarrage	38
13.4 Variables d'environnement	39
14 · Scalabilité	40
14.1 État actuel : une instance unique	40
14.2 Ce qui facilite la montée en charge	40
14.3 Goulots d'étranglement identifiés	40
14.4 Trajectoire d'évolution	41
15 · Stratégie de tests	42
15.1 Organisation par couche	42
15.2 Le principe des mocks	42
15.3 Un patron Vitest important : vi.hoisted()	42
15.4 Couverture	42
16 · Points forts, points faibles et feuille de route	44

16.1 Points forts	44
16.2 Points faibles	44
16.3 Feuille de route priorisée	44
17 · Conclusion	45
Annexe A · Correspondance avec le référentiel RNCP 5 DWWM	46
Bloc 1 — Développer la partie front-end sécurisée	46
Bloc 2 — Développer la partie back-end sécurisée	46
Annexe B · Retour façon jury	47
B.1 Points forts à valoriser	47
B.2 Points faibles à anticiper	47
B.3 Questions probables et éléments de réponse	47
Annexe C · Glossaire	49
Annexe D · Référence des routes API	50
Annexe E · Variables d'environnement	51
Annexe F · Liens avec la formation Holberton	52

0 - Préambule et résumé exécutif

Ce document constitue la documentation technique complète de **TripGenie**, une application web de génération de voyages personnalisés pilotée par intelligence artificielle. Il a été rédigé dans un double objectif : servir de support à la **démonstration des fondamentaux** de Holberton School, et constituer le **dossier de projet (DP)** du titre professionnel **RNCP niveau 5 — Développeur Web et Web Mobile (DWWM)**.

Contrairement à un simple manuel d'utilisation, ce rapport adopte le point de vue de l'ingénieur : chaque choix d'architecture, de langage, de framework et de service externe y est non seulement décrit, mais **justifié** par rapport aux alternatives écartées. L'objectif est de démontrer une maîtrise réelle des concepts, et non une simple capacité à assembler des briques techniques.

0.1 À propos de ce document

Le lecteur trouvera dans les pages qui suivent :

- Une présentation du problème métier et de la proposition de valeur de l'application ;
- L'architecture générale du système (frontend, backend, base de données, services externes) appuyée par des schémas ;
- La justification détaillée de chaque choix technologique, comparé à ses alternatives ;
- Le fonctionnement du pipeline d'intelligence artificielle et de l'algorithme de scoring ;
- Le modèle de données relationnel et le schéma entité-association ;
- La stratégie de sécurité, d'authentification et la défense en profondeur mise en place ;
- La conteneurisation Docker, le déploiement en production et l'analyse de scalabilité ;
- Une analyse honnête des **points forts et des points faibles**, accompagnée d'une feuille de route ;
- En annexe : la correspondance avec le référentiel RNCP, un retour façon jury, un glossaire et les références d'API.

0.2 Résumé exécutif

TripGenie répond à un constat simple : les plateformes de voyage classiques (Booking, Kayak, TripAdvisor) sont des **agrégateurs de données brutes**. Elles renvoient des centaines de résultats que l'utilisateur doit trier seul. TripGenie inverse la logique : l'utilisateur décrit son voyage en langage naturel, et l'application produit un **pack complet clé en main** — vols, hébergement, itinéraire jour par jour, activités, restaurants réels, météo et budget ventilé — adapté à un mode de voyage choisi (fête, luxe, étudiant, groupe, détente, surprise).

Techniquement, l'application est une **Single Page Application** React/TypeScript adossée à une **API REST** Express/TypeScript, le tout en **full-stack TypeScript**. Le cœur du produit est un **pipeline d'IA orchestré côté serveur** : des recherches web temps réel sont lancées en parallèle, un grand modèle de langage assemble un pack structuré, puis un algorithme déterministe le note. La persistance s'appuie sur **PostgreSQL** hébergé via **Supabase**.

Chiffres clés du projet

Full-stack **TypeScript** (serveur .ts, client .tsx) · **6 modes** de voyage · pipeline IA à **5 niveaux de repli** LLM (Ollama → Gemini → Claude → OpenRouter → Mocks) · **6 tables** PostgreSQL · **8+ services externes** intégrés avec dégradation gracieuse · suite de **~17 fichiers de tests** Vitest + Supertest · déploiement PaaS sur Render + conteneurisation Docker pour la portabilité.

0.3 Fiche signalétique du projet

Attribut	Valeur
Nom	TripGenie
Nature	Application web full-stack (SPA + API REST)
Domaine	Voyage / planification assistée par IA
Langage principal	TypeScript (frontend et backend)
Frontend	React 18, Vite 5, React Router 6, Zustand 5, React Query 5, Tailwind 3
Backend	Node.js ≥18, Express 4, Zod 4, JWT, bcryptjs, Helmet
Base de données	PostgreSQL (hébergée sur Supabase)
IA / LLM	Gemini, Claude, OpenRouter, Ollama — avec repli en cascade
Tests	Vitest 4 + Supertest (~17 fichiers)
Déploiement	Render (PaaS, runtime Node natif) + Docker (portabilité)
Auteur	Alexis Laubert
Cadre	Holberton School / RNCP 5 DWWM

Tableau 0.1 — Carte d'identité technique de TripGenie.

0.4 Conventions de lecture

Pour faciliter la lecture, ce document utilise un jeu d'encadrés typés :

Encadré « point fort »

Met en valeur une décision technique solide et défendable à l'oral.

Encadré « point faible »

Signale honnêtement une limite, une dette technique ou un risque assumé. La transparence sur ces points est attendue par un jury.

Encadré « attention »

Souligne un piège, une subtilité ou un compromis qui mérite explication.

Le code en police à chasse fixe désigne un nom de fichier, une variable, une route ou une commande. Les blocs de code sont présentés sur fond grisé.

1 · Présentation du projet

1.1 Le problème

Organiser un voyage est une tâche cognitivement coûteuse. Un utilisateur qui souhaite partir doit jongler entre plusieurs plateformes spécialisées : un comparateur de vols, un site de réservation d'hôtels, un guide d'activités, une application météo, sans parler de la recherche d'événements locaux et de restaurants. Chacun de ces sites fonctionne comme un **agrégateur** : il maximise le nombre de résultats affichés plutôt que la pertinence de la décision finale.

Résultat : l'utilisateur se retrouve face à 300 hôtels, 80 vols et des centaines d'activités, sans aide pour les **combiner de façon cohérente** en fonction de son budget, de ses dates et de son style de voyage. La charge de synthèse repose entièrement sur lui.

1.2 La proposition de valeur

TripGenie réalise cette synthèse à la place de l'utilisateur. À partir d'une description en langage naturel — ou d'un court échange conversationnel d'onboarding — l'application génère un **pack de voyage complet et cohérent** :

- **Vols** avec fourchette de prix par personne et liens profonds vers les comparateurs ;
- **Hébergement** adapté au mode et au budget ;
- **Itinéraire jour par jour** structuré ;
- **Activités et restaurants réels** issus d'API de données géolocalisées (Foursquare, Yelp, PredictHQ) ;
- **Météo** temps réel sur la période du séjour ;
- **Budget ventilé** par poste (vols, hébergement, activités, restauration, transport) ;
- Un **score de pertinence** déterministe qui évalue l'adéquation du pack au mode choisi.

La valeur ajoutée n'est donc pas la donnée brute — disponible partout — mais la **mise en cohérence** de cette donnée au service d'une décision, via un pipeline IA + un scoring multi-critères.

1.3 Public cible

- **Voyageurs de loisir** qui veulent gagner du temps et déléguer la phase de recherche ;
- **Groupes d'amis** qui souhaitent décider ensemble (fonction de votes consensus par élément) ;
- **Étudiants** au budget contraint, recherchant le meilleur rapport expérience/prix ;
- **Voyageurs « luxe »** ou « fête » dont les attentes très spécifiques sont mal servies par les agrégateurs génériques.

1.4 Parcours utilisateur type

1. L'utilisateur arrive sur la page d'accueil et décrit son envie de voyage, soit via le formulaire, soit via le chatbot d'onboarding.
2. Le chatbot extrait progressivement les informations clés (destination, dates, voyageurs, budget, mode) en quelques tours.
3. Une fois les informations suffisantes réunies, l'utilisateur lance la génération.

4. Le serveur exécute le pipeline IA : recherches parallèles, assemblage du pack par le LLM, fusion des restaurants, scoring.
5. Le pack complet s'affiche : vols, hôtel, itinéraire, activités, carte interactive, budget en graphique, météo.
6. L'utilisateur peut affiner le pack par **conversation** (« remplace l'hôtel par moins cher »), partager le voyage, ou voter en groupe.
7. S'il est connecté, le voyage est sauvegardé et retrouvable dans sa liste de voyages.

1.5 Périmètre fonctionnel

Domaine	Fonctionnalités
Authentification	Inscription, connexion, déconnexion, session par cookie httpOnly
Génération IA	Onboarding conversationnel, suggestion de destinations, génération de pack complet
Modification	Chat de modification post-génération (partie agentique)
Voyages	Création, consultation, liste, suppression, partage public
Collaboration	Votes consensus par élément, gestion de collaborateurs
Préférences	Mode par défaut, ville de départ, centres d'intérêt, devise
Données réelles	Vols, hôtels, événements, restaurants, météo, photos

Tableau 1.1 — Périmètre fonctionnel couvert par l'application.

1.6 Ce que TripGenie n'est pas

Il est important, en particulier pour l'oral, de cadrer précisément la nature du système. TripGenie **n'est pas un agent IA autonome**. Il ne décide pas seul de la suite d'outils à appeler ni n'itère librement vers un objectif. Il repose sur un pipeline dont les étapes sont **prédéfinies et ordonnées**.

« TripGenie repose sur un pipeline IA orchestré côté serveur, avec une composante conversationnelle agentique uniquement pour la modification post-génération. »

Cette distinction est développée au chapitre 6. Elle évite une sur-promesse marketing (« agent autonome ») et reflète fidèlement l'implémentation réelle — une honnêteté technique appréciée par un jury.

2 · Méthodologie et gestion de projet

2.1 Démarche de développement

Le projet a été mené de façon **itérative et incrémentale**. Plutôt que de spécifier l'ensemble du système en amont, chaque fonctionnalité a été développée comme une tranche verticale complète — de la route Express jusqu'au composant React — puis testée et stabilisée avant de passer à la suivante. Cette approche, proche de l'esprit agile, permet d'avoir en permanence une application fonctionnelle et de réorienter les priorités selon les difficultés rencontrées.

L'ordre de construction a suivi la dépendance naturelle des briques : d'abord le socle (authentification, base de données, structure de l'API), puis le pipeline IA, puis l'enrichissement par les services externes, et enfin le raffinement de l'interface et la couverture de tests.

2.2 Organisation du dépôt

Le code est organisé en **monorepo** : le frontend et le backend cohabitent dans un même dépôt Git, sous deux dossiers distincts. Ce choix simplifie la cohérence des versions, le partage de types TypeScript entre client et serveur, et le déploiement en un seul service.

```
tripgenie/
├── server/                                Backend Node.js / Express (TypeScript)
│   ├── index.ts                          Point d'entree : middlewares, routes, démarrage
│   ├── routes/                           auth, trips, ai, votes, photos, packs,
│   │   │                                   preferences, collaborators
│   │   └── middleware/                   auth (requireAuth/optionalAuth), limiter
│   │   └── services/
│   │       ├── claude/                  Pipeline IA : core, analyze, pack, chat
│   │       ├── smartSearch.ts          Vols / hotels / evenements (Tavily + IA)
│   │       ├── foursquare.ts           Restaurants reels (chaîne de repli Yelp)
│   │       ├── predictHQ.ts            Evenements reels (concerts, festivals)
│   │       ├── scoring.ts              Scoring multi-criteres deterministe
│   │       ├── weather.ts              Meteo temps reel (OpenWeatherMap)
│   │       ├── photo.ts                Photo destination (Unsplash -> Pexels)
│   │       ├── providers.ts            Abstraction multi-LLM
│   │       └── mocks.ts                Données de repli ultime
│   ├── lib/                             AppError, constants, types
│   └── db/                             supabase.ts (client singleton)
├── client-react/                         Frontend React / Vite (TypeScript)
│   ├── src/
│   │   ├── pages/                       Home, Trips, TripDetail, Login, Preferences
│   │   ├── components/                  layout, results, chat, ui
│   │   ├── store/                       index.ts (4 stores Zustand)
│   │   └── lib/                         api.ts (requêtes HTTP)
│   ├── tests/                           Vitest + Supertest (unit, services,
│   │   │                                   security, integration)
│   ├── Dockerfile                       Image de production multi-stage
│   ├── docker-compose.yml               Orchestration locale
│   └── render.yaml                      Configuration de déploiement Render
```

2.3 Outils et environnement

Outil	Rôle dans le projet
Git	Gestion de versions, commits atomiques en français
npm	Gestion des dépendances et scripts (dev, build, test)
tsx / nodemon	Exécution TypeScript directe + rechargement à chaud en dev
Vite	Serveur de développement frontend + build de production
Vitest	Exécution des tests unitaires et d'intégration
dotenv	Chargement des variables d'environnement locales
Docker	Conteneurisation pour portabilité et reproductibilité

Tableau 2.1 — Chaîne d'outils du projet.

2.4 Conventions de commit

Les commits suivent une convention simple, en français, un sujet par commit :

- Format type: description courte — types : feat, fix, refactor, test, docs, chore ;
- Exemples : feat: ajouter proxy Unsplash cote serveur, fix: corriger acces issues Zod v4 ;
- Chaque commit est **atomique** : il représente une unité de changement cohérente et testable.

3 - Architecture générale

3.1 Vue d'ensemble

TripGenie suit une architecture **trois tiers** classique mais adaptée aux contraintes d'une application pilotée par IA :

- **Tier présentation** — une SPA React qui s'exécute dans le navigateur ;
- **Tier logique** — une API REST Express qui orchestre l'authentification, le pipeline IA et l'accès aux services externes ;
- **Tier données** — une base PostgreSQL gérée par Supabase, et un ensemble d'API tierces fournissant les données réelles (vols, restaurants, météo...).

La particularité de TripGenie réside dans l'épaisseur du tier logique : ce n'est pas un simple CRUD, mais un véritable **orchestrateur** qui coordonne des appels parallèles à de multiples fournisseurs, gère leurs échecs, et synthétise le résultat via un LLM.

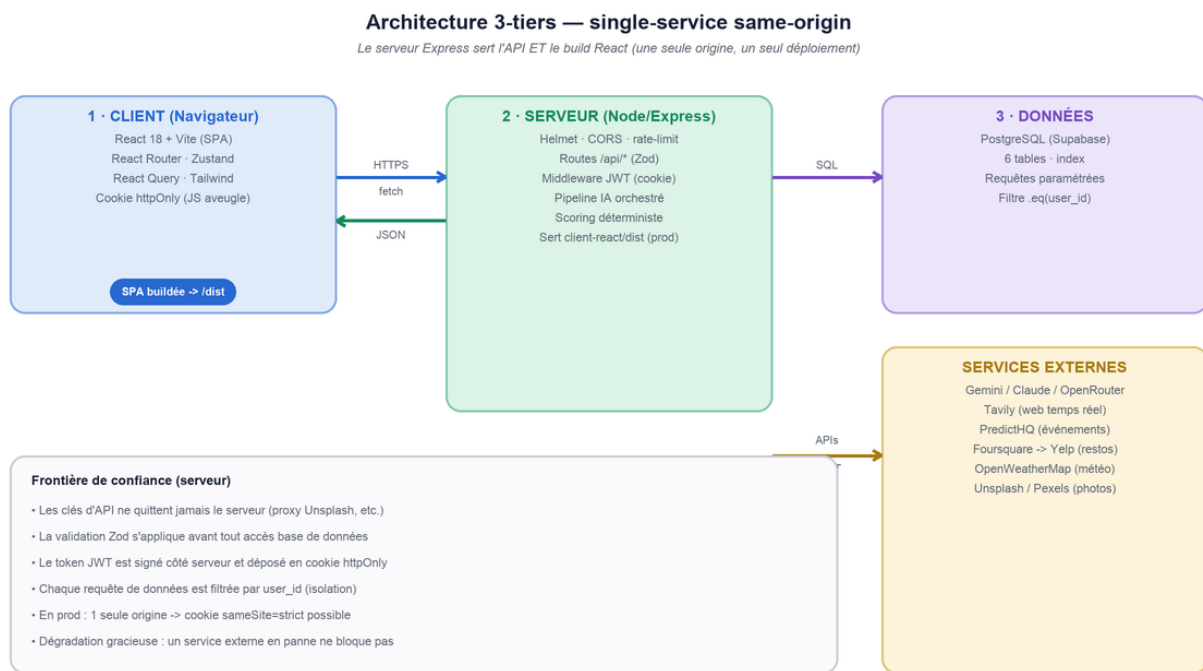


Figure 3.1 — Architecture générale : navigateur, API Express, pipeline IA, base de données et services externes.

3.2 Le choix d'un service unique (same-origin)

En production, l'application est déployée comme **un seul service**. Le serveur Express sert à la fois l'API (sous /api) et les fichiers statiques du build React (le dossier dist du client). Frontend et backend partagent donc la **même origine**.

Ce choix a une conséquence de sécurité directe et déterminante : il permet d'utiliser un cookie d'authentification avec l'attribut sameSite: 'strict'. Si le frontend et l'API étaient sur deux domaines différents, ce réglage strict casserait l'authentification et obligerait à un compromis (sameSite: 'none') plus exposé au CSRF.

Décision défendable

Le déploiement same-origin n'est pas qu'une simplification d'infrastructure : c'est ce qui rend possible le cookie `httpOnly + sameSite:strict`, pierre angulaire de la défense contre le vol de session et le CSRF (voir chapitre 10).

3.3 Cycle de vie d'une requête

Toute requête entrante traverse une chaîne ordonnée de middlewares avant d'atteindre sa route. Cet ordre n'est pas anodin : les middlewares de sécurité et de limitation de débit sont placés en amont, de sorte qu'une requête abusive est rejetée le plus tôt possible, avant de consommer des ressources.

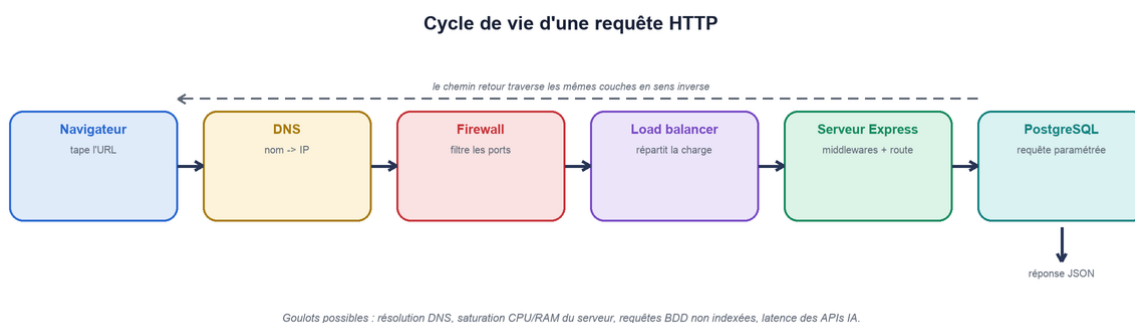


Figure 3.2 — Cycle de vie d'une requête HTTP, des middlewares de sécurité jusqu'au gestionnaire d'erreurs global.

À l'autre extrémité, un **gestionnaire d'erreurs global** centralise le traitement des exceptions. Les erreurs métier sont encapsulées dans une classe `AppError` qui porte un code HTTP, garantissant des réponses cohérentes (400, 401, 404, 409, 429...) sans fuite de détails internes vers le client.

3.4 Séparation des responsabilités

Le backend applique une séparation stricte en couches, transposée des principes vus en formation (architecture en couches de HBnB) :

- **Routes** — valident l'entrée (Zod), orchestrent, formatent la réponse HTTP. Aucune logique métier lourde ;
- **Services** — contiennent la logique métier réutilisable (pipeline IA, scoring, recherches externes) ;
- **Accès données** — isolé dans le client Supabase (`db/supabase.ts`), point unique de contact avec la base.

Cette séparation rend chaque couche testable indépendamment : les tests unitaires ciblent les services sans HTTP, les tests d'intégration ciblent les routes avec services mockés.

4 · Choix technologiques et justifications

Ce chapitre est le cœur de la démonstration d'ingénierie. Chaque technologie retenue y est confrontée à ses alternatives crédibles. Le fil conducteur est constant : **réduire le coût cognitif et le risque** pour un développeur unique, tout en gardant une trajectoire de montée en charge crédible.

4.1 TypeScript de bout en bout

L'intégralité de l'application est écrite en **TypeScript**, côté serveur (.ts) comme côté client (.tsx). Ce choix répond à deux questions de jury fréquentes : pourquoi pas du JavaScript simple, et pourquoi pas Python comme en formation ?

Pourquoi pas JavaScript ?

JavaScript n'offre aucune vérification de type avant l'exécution. Sur une application qui manipule des objets complexes — un « pack » de voyage imbriqué sur plusieurs niveaux — l'absence de typage transforme la moindre faute de frappe sur une clé en bug silencieux découvert en production. TypeScript déplace cette classe d'erreurs **du runtime vers la compilation** : le compilateur refuse le code avant qu'il ne s'exécute.

Pourquoi pas Python / Flask ?

La formation Holberton s'appuie largement sur Python (Flask, HBnB). Le choix de Node/TypeScript est délibéré : **un seul langage sur toute la chaîne**. Le frontend, le backend et même les types de données partagés sont écrits dans le même langage. Il n'y a aucun changement de contexte mental entre le client et le serveur, aucune duplication de modèles de données entre deux écosystèmes, et un seul outillage à maîtriser. Flask aurait imposé un runtime séparé, une sérialisation de part et d'autre, et un déploiement distinct.

Argument central

Le full-stack TypeScript permet de **partager les types entre client et serveur**. Le type d'un Pack défini côté serveur est importé tel quel côté React. Le contrat de données est unique : si le serveur change la forme d'un objet, le client ne compile plus. C'est une garantie qu'aucune architecture bi-langage ne peut offrir aussi simplement.

4.2 Node.js + Express

Express 4 est un framework HTTP minimaliste fondé sur le patron *middleware*. Chaque préoccupation transversale (sécurité, journalisation, limitation de débit, authentification) est un middleware enfichable dans une chaîne ordonnée. Cette granularité est pédagogiquement précieuse : elle rend visible et explicite chaque étape du traitement d'une requête, là où un framework « tout-en-un » masquerait ces mécanismes.

Le choix d'Express plutôt qu'un framework plus opiniâtre (NestJS, par exemple) se justifie par l'échelle du projet : NestJS apporte une structure lourde (modules, injection de dépendances, décorateurs) dont le bénéfice n'apparaît qu'à partir d'équipes et de bases de code importantes. Pour un développeur unique, cette structure serait un coût sans retour.

4.3 React + Vite — pourquoi pas Next.js ?

Le frontend est une **Single Page Application** React servie par **Vite**. Vite a été préféré à l'ancien Create React App pour son serveur de développement quasi instantané (rechargement à chaud sur modules ES natifs) et son build de production optimisé.

La question la plus probable du jury est : pourquoi ne pas avoir utilisé Next.js, le framework React de référence ? La réponse tient à la nature du produit :

- **Next.js excelle sur le rendu côté serveur (SSR) et le SEO.** Or TripGenie est une application **derrière authentification** : Google ne peut pas se connecter, donc le référencement n'a aucune valeur ici ;
- L'application maintient un **état riche en mémoire** pendant des opérations longues (génération IA de ~15 s, chat de modification). Une SPA conserve naturellement ce contexte sans rechargement ;
- Séparer clairement React (présentation) et Express (API) est **pédagogiquement plus riche** : cela montre explicitement la frontière frontend / backend / base de données, au lieu de la fondre dans un framework unique.

Position nuancée

Le choix n'est pas dogmatique : pour un site vitrine public ou une boutique e-commerce où le SEO est vital, Next.js (SSR/ISR) serait le bon choix. Savoir **quand** un outil est pertinent vaut mieux que de l'adopter par réflexe.

4.4 Zustand + React Query — pourquoi pas Redux ?

La gestion d'état distingue deux natures de données, souvent confondues :

- L'**état client** (formulaire en cours, thème, utilisateur connecté) — géré par **Zustand** ;
- L'**état serveur** (les voyages, les votes, les préférences récupérés via l'API) — géré par **React Query**.

Redux aurait été surdimensionné. Il impose une cérémonie importante (actions, reducers, dispatch, store central) et, surtout, traite l'état serveur comme de l'état client — obligeant à écrire manuellement la logique de cache, de rechargement et d'invalidation. React Query résout ce problème nativement : cache, états de chargement/erreur, refetch et invalidation sont fournis. Zustand, de son côté, offre une API minimaliste (un hook, pas de *boilerplate*) pour le peu d'état purement client nécessaire.

Les quatre stores Zustand du projet (`useSearchStore`, `useChatStore`, `useAuthStore`, `useThemeStore`) utilisent le middleware `persist` avec `partialize` pour ne persister que le strict nécessaire dans le `localStorage` (par exemple, le store d'auth ne persiste que l'objet `user`, jamais de jeton — celui-ci vit dans un cookie `httpOnly` inaccessible au JavaScript).

4.5 PostgreSQL + Supabase — pourquoi pas MongoDB ni Firebase ?

Pourquoi du relationnel (PostgreSQL) ?

Les données de TripGenie sont **fortement relationnelles** : un utilisateur possède des voyages, un voyage possède des packs, un pack reçoit des votes, un voyage a des collaborateurs. Ces relations un-à-plusieurs, avec suppression en cascade, sont l'archétype du modèle relationnel. PostgreSQL garantit l'intégrité référentielle (clés étrangères) que MongoDB, orienté documents, ne fournit pas nativement.

Le pack de voyage lui-même, en revanche, est un objet semi-structuré et volumineux. Il est stocké dans une colonne **JSONB** : on bénéficie ainsi du meilleur des deux mondes — relations strictes pour le modèle, souplesse documentaire pour la charge utile IA. C'est précisément ce qu'un MongoDB pur ne permet pas

d'articuler proprement.

Pourquoi Supabase et pas Firebase ?

Supabase est utilisé ici comme **hébergeur PostgreSQL + client SQL**, pas comme plateforme propriétaire. La base reste du PostgreSQL standard : aucune dépendance forte à un fournisseur, migration possible vers n'importe quel hébergeur Postgres. Firebase (Firestore), à l'inverse, est une base NoSQL propriétaire qui enferme le modèle de données et la logique de requête dans son écosystème.

Compromis assumé

Ni Supabase Auth ni le Row Level Security (RLS) ne sont utilisés : l'authentification est gérée au niveau applicatif (JWT custom), et chaque requête filtre explicitement par `user_id`. Ce point — un compromis documenté, pas un oubli — est détaillé au chapitre 10.

4.6 Zod — la validation comme contrat

Zod valide toutes les entrées des routes sensibles avant qu'elles n'atteignent la logique métier ou la base. Un schéma Zod est déclaratif et lisible, produit des messages d'erreur précis, et — atout décisif en TypeScript — **infère automatiquement le type** des données validées. Le schéma de validation et le type statique ne font qu'un : impossible qu'ils divergent.

```
const createTripSchema = z.object({
  destination: z.string().min(1).max(100),
  mode:        z.enum(MODES_LIST),
  travelers:   z.number().int().min(1).max(50).optional(),
  budget:      z.union([z.string(), z.number()]).optional(),
});
// safeParse -> soit { success:true, data } typé, soit { success:false, error }
```

4.7 Tailwind CSS

Tailwind applique des classes utilitaires directement dans le balisage. L'avantage pour un développeur unique est la **cohérence visuelle sans changement de contexte** : pas de va-et-vient entre un fichier CSS et le composant, pas de noms de classes à inventer, et une purge automatique qui ne livre en production que les classes réellement utilisées. Le responsive (mobile/desktop) est géré par de simples préfixes (`lg:`), ce qui a permis de corriger efficacement les problèmes d'affichage mobile (chat en *bottom sheet*, accessibilité clavier).

4.8 Vitest — pourquoi pas Jest ?

Vitest est le moteur de test. Il a été choisi face à Jest pour une raison structurelle : le projet est entièrement bâti sur les **modules ES** et l'outillage Vite. Vitest est nativement compatible ESM et réutilise la configuration Vite existante. Jest aurait nécessité une couche de transpilation supplémentaire pour gérer les imports ES. Vitest est par ailleurs sensiblement plus rapide et partage la même API que Jest (`describe`, `it`, `expect`), sans coût d'apprentissage.

4.9 Synthèse des arbitrages

Besoin	Choix retenu	Alternative écartée	Raison principale
Langage	TypeScript	JavaScript / Python	Typage statique + un seul langage full-stack
API	Express	NestJS	Minimalisme adapté à un dev unique
Frontend	React + Vite	Next.js	App authentifiée : SEO sans valeur
État client	Zustand	Redux	API minimaliste, pas de boilerplate
État serveur	React Query	Redux + thunks	Cache et refetch natifs
Base	PostgreSQL	MongoDB	Données relationnelles + JSONB
Hébergeur DB	Supabase	Firebase	Postgres standard, pas d'enfermement
Validation	Zod	Joi / manuel	Inférence de type intégrée
Tests	Vitest	Jest	ESM natif, vitesse, config Vite

Tableau 4.1 — Récapitulatif des choix technologiques et des alternatives écartées.

5 · Le backend en détail

5.1 Point d'entrée et chaîne de middlewares

Le fichier `server/index.ts` est le point d'entrée du backend. Il assemble, dans un ordre précis, la chaîne de middlewares que toute requête traverse, monte les routeurs sous le préfixe `/api`, sert les fichiers statiques du build React en production, puis installe le gestionnaire d'erreurs global en dernier.

L'ordre des middlewares suit une logique de **défense en profondeur** : on filtre et sécurise avant de traiter.

```
app.use(helmet()) // 1. En-têtes de securite HTTP
app.use(cors({ origin, credentials:true })) // 2. Origines autorisees
app.use(express.json()) // 3. Parsing du corps JSON
app.use(cookieParser()) // 4. Lecture des cookies httpOnly
app.use(morgan('dev')) // 5. Journalisation des requetes

app.use('/api/auth', authRouter) // Routeurs metier
app.use('/api/ai', aiRouter)
app.use('/api/trips', tripsRouter)
// ... votes, photos, preferences, collaborators

app.use(express.static(clientDist)) // SPA React (production)
app.use(globalErrorHandler) // En dernier : capture tout
```

5.2 Les middlewares de sécurité

Middleware	Rôle
Helmet	Pose des en-têtes HTTP de sécurité (X-Frame-Options, CSP, HSTS...) pour réduire la surface d'attaque (clickjacking, sniffing MIME)
CORS	Restreint les origines autorisées à appeler l'API ; <code>credentials:true</code> autorise l'envoi du cookie
cookie-parser	Rend les cookies lisibles côté serveur (lecture du jeton JWT <code>httpOnly</code>)
express.json	Analyse le corps JSON des requêtes entrantes
Morgan	Journalise chaque requête (méthode, route, statut, durée) — colorisé en mode dev

Tableau 5.1 — Middlewares transversaux et leur rôle.

5.3 Limitation de débit (rate limiting)

La limitation de débit protège l'API contre le spam, les attaques par force brute et l'épuisement de ressources — un enjeu **financier direct** ici, car chaque appel au pipeline IA déclenche des requêtes payantes vers des LLM et des API externes. Plutôt qu'une limite globale unique, le projet applique des limiteurs **dédiés aux routes coûteuses**, calibrés selon leur coût réel.

Route	Limite	Justification
<code>/api/ai/generate</code>	10 requêtes / heure / IP	Génération très coûteuse (LLM + multiples API)
<code>/api/ai/chat</code>	30 requêtes / 15 min / IP	Modification conversationnelle, coût modéré

Route	Limite	Justification
/api/votes	10 requêtes / minute / IP	Limite anti-spam sur l'écriture des votes

Tableau 5.2 — Limiteurs de débit appliqués aux routes sensibles.

Point faible identifié

Il n'existe **pas de limiteur global** couvrant l'ensemble des routes (par exemple l'authentification). Une route comme `/api/auth/login` gagnerait à être protégée par un limiteur dédié contre le bourrage d'identifiants (*credential stuffing*). C'est une amélioration prioritaire de la feuille de route (chapitre 16).

5.4 Gestion centralisée des erreurs

Toutes les erreurs convergent vers un **gestionnaire global** monté en dernier. Les erreurs métier sont levées sous forme d'instances `AppError`, une classe custom qui transporte un message et un **code HTTP**. Le gestionnaire global lit ce code et formate une réponse JSON homogène, sans jamais divulguer de détail technique interne (trace de pile, requête SQL) au client.

```
class AppError extends Error {
  statusCode: number;
  constructor(message: string, statusCode = 500) {
    super(message);
    this.statusCode = statusCode;
  }
}

// Dans une route : return next(new AppError('Voyage introuvable', 404));
// Le handler global renvoie : res.status(404).json({ error: '...' })
```

Ce patron garantit que le **format de réponse d'erreur est unique** sur toute l'API, ce qui simplifie la gestion d'erreurs côté client (la fonction `request()` du frontend lit toujours `data.error`).

5.5 Les routes REST

L'API respecte les conventions REST : ressources nommées au pluriel, verbes HTTP porteurs de sens (GET pour lire, POST pour créer, PUT pour mettre à jour, DELETE pour supprimer), et codes de statut appropriés.

Méthode + route	Rôle	Accès
POST /api/auth/signup	Création de compte → 201 + cookie	Public
POST /api/auth/login	Connexion → 200 + cookie	Public
POST /api/auth/logout	Déconnexion → efface le cookie	Public
GET /api/auth/me	Profil de l'utilisateur connecté	Auth
POST /api/ai/generate	Génère un pack complet	Auth optionnelle
POST /api/ai/chat	Modifie un pack (agentique)	Auth optionnelle
POST /api/ai/onboarding	Onboarding conversationnel	Public
GET /api/trips	Liste des voyages de l'utilisateur	Auth
GET /api/trips/:id	Détail d'un voyage	Auth

Méthode + route	Rôle	Accès
GET /api/trips/share/:id	Voyage en accès partagé	Public
DELETE /api/trips/:id	Suppression d'un voyage	Auth
POST /api/votes	Vote sur un élément d'un pack	Public + limité
GET /api/preferences	Préférences utilisateur	Auth
GET /api/photos/:city	Proxy photo (clé masquée)	Public

Tableau 5.3 — Principales routes de l'API REST.

5.6 Sémantique des codes HTTP

Le respect de la sémantique des codes de statut fait partie des fondamentaux REST :

- **200 OK** — requête réussie (lecture, mise à jour) ;
- **201 Created** — ressource créée (inscription, création de voyage) ;
- **400 Bad Request** — requête mal formée (échec de validation Zod) ;
- **401 Unauthorized** — authentification manquante ou invalide (jeton absent, expiré, mal signé) ;
- **404 Not Found** — ressource inexistante *ou* appartenant à un autre utilisateur (protection contre l'IDOR) ;
- **409 Conflict** — conflit avec l'état actuel : typiquement un email déjà enregistré. À distinguer du 400 — la donnée est valide, c'est son unicité qui pose problème ;
- **429 Too Many Requests** — limite de débit dépassée.

6 · Le pipeline d'intelligence artificielle

6.1 Pipeline orchestré contre agent autonome

La distinction est fondamentale et doit être maîtrisée à l'oral. Un **agent autonome** choisit lui-même quels outils appeler, dans quel ordre, et peut itérer librement jusqu'à atteindre un but. TripGenie ne fonctionne pas ainsi : ses étapes sont **prédéfinies et s'enchaînent toujours dans le même ordre**. C'est un *pipeline orchestré*.

« Les étapes sont fixes : valider → rechercher en parallèle → assembler → fusionner → noter → sauvegarder. Toujours cet ordre, toujours ces étapes. »

La seule brique réellement agentique est le **chat de modification** (§6.6), où le modèle décide dynamiquement quoi changer dans un pack existant. Présenter le système comme un « pipeline orchestré avec une composante conversationnelle agentique » est plus honnête, et plus juste techniquement, que la sur-promesse « agent IA autonome ».

6.2 Les étapes du pipeline de génération

La route POST `/api/ai/generate` déclenche la séquence suivante :

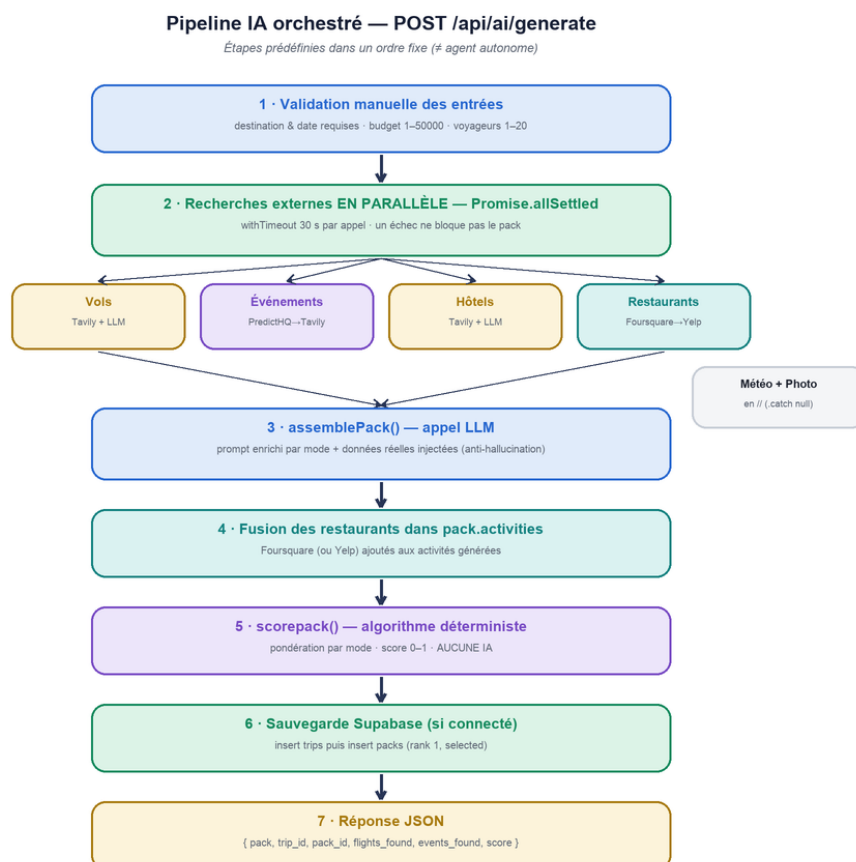


Figure 6.1 — Pipeline de génération : validation, recherches parallèles, assemblage LLM, fusion des restaurants, scoring, sauvegarde.

1. **Validation** des paramètres d'entrée (destination, origine, dates, voyageurs, budget, mode) ;
2. **Recherches en parallèle** via `Promise.allSettled` : vols, événements, hôtels, météo, photo de destination ;
3. **Recherche de restaurants** en parallèle également, avec chaîne de repli Foursquare → Yelp → liste vide ;
4. **Assemblage du pack** par le LLM : génération d'un JSON structuré adapté au mode, via ingénierie de prompt ;
5. **Fusion** des restaurants réels dans les activités du pack ;
6. **Scoring** déterministe du pack selon la pondération du mode ;
7. **Sauvegarde** dans PostgreSQL si l'utilisateur est connecté ;
8. **Réponse JSON** : pack, score, nombre de vols et d'événements trouvés.

6.3 Promise.allSettled — un choix technique structurant

Les cinq recherches externes sont lancées **simultanément**, pas séquentiellement. Le choix de `Promise.allSettled` plutôt que `Promise.all` est délibéré et constitue un point de discussion classique en entretien.

```
const results = await Promise.allSettled([
  smartFlightSearch(...), // Tavily : vols
  smartEventsSearch(...), // PredictHQ ou Tavily : evenements
  smartHotelSearch(...), // Tavily : hotels
  getRealWeather(...), // OpenWeatherMap
  getDestinationPhoto(...) // Unsplash -> Pexels
]);
```

Pourquoi allSettled et pas all ?

`Promise.all` **rejette dès qu'une seule promesse échoue** : si l'API météo tombe, toute la génération s'arrête. `Promise.allSettled` attend la résolution de **toutes** les promesses, quel que soit leur sort. Le pack est alors construit avec les données disponibles, en se repliant sur des données simulées pour les sources défaillantes. C'est le fondement de la **dégradation gracieuse** du système.

6.4 Repli multi-LLM en cascade

L'appel au modèle de langage n'est jamais adressé à un fournisseur unique. Une fonction d'abstraction tente les fournisseurs **en cascade** : si le premier échoue (clé absente, quota dépassé, panne), elle bascule automatiquement sur le suivant. En dernier recours, des données simulées (« mocks ») garantissent qu'une réponse est toujours produite.

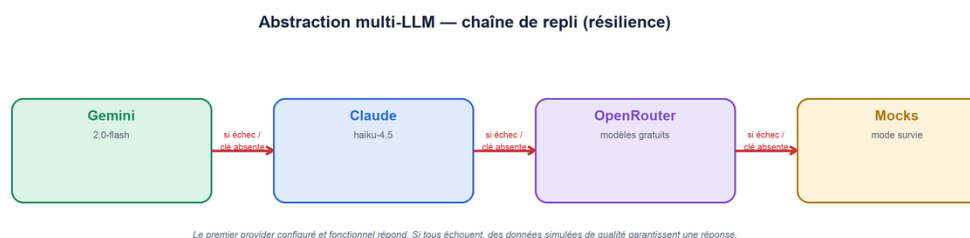


Figure 6.2 — Cascade de repli multi-LLM : du fournisseur local jusqu'aux données simulées.

Ordre	Fournisseur	Rôle
1	Ollama (si activé)	LLM local — coût nul, vie privée, optionnel
2	Google Gemini	LLM principal (modèle 2.0-flash)
3	Anthropic Claude	LLM secondaire (modèle Haiku)
4	OpenRouter	Accès à des modèles gratuits multiples
5	Mocks	Données simulées — garantit une réponse, jamais d'échec total

Tableau 6.1 — Ordre de repli des fournisseurs de LLM.

Cette résilience à plusieurs niveaux signifie que **l'application reste fonctionnelle même sans aucune clé d'API IA configurée** : elle bascule alors en « mode survie » sur les mocks. C'est précieux en développement, en démonstration et en cas d'incident fournisseur.

6.5 L'assemblage du pack : ingénierie de prompt

Le cœur qualitatif du système est la fonction `assemblePack()`. Elle construit un prompt riche, conditionné par le **mode de voyage** :

- **Nombre de nuits** calculé depuis les dates, ou estimé depuis le budget si les dates manquent (borné entre 2 et 14) ;
- **Budget par personne** dérivé du budget total divisé par le nombre de voyageurs, classé par paliers de gamme ;
- **Persona de mode** : chaque mode (fête, luxe, étudiant...) injecte un ton et des consignes différentes ;
- **Consignes d'activités** spécifiques : le mode « fête » privilégie la vie nocturne, le mode « luxe » oriente vers le haut de gamme, etc.

Point faible identifié

Dans la génération des vols, l'aéroport et la ville de départ sont **codés en dur** (`from: 'CDG', from_city: 'Paris'`), indépendamment de l'origine réellement saisie par l'utilisateur. Un utilisateur partant de Bordeaux verra malgré tout un vol au départ de Paris. C'est un bug fonctionnel honnête à corriger : la ville d'origine doit être propagée jusqu'au générateur de vols.

6.6 La partie agentique : le chat de modification

La route `POST /api/ai/chat` est la seule composante véritablement agentique. L'utilisateur formule une demande en langage naturel (« remplace l'hôtel par quelque chose de moins cher ») accompagnée du pack courant. Le modèle décide **lui-même** quels éléments du pack modifier, sans étapes prédéfinies, et renvoie un objet de modifications — éventuellement assorti d'un indicateur de régénération complète si la demande est trop profonde. En cas d'erreur, le système se replie là aussi sur un mode dégradé.

7 · Intégrations de services externes

7.1 Philosophie : la dégradation gracieuse

TripGenie dépend d'au moins huit services tiers. Le principe directeur de toute l'intégration est qu'**aucune dépendance externe ne doit pouvoir faire échouer la génération**. Chaque appel externe est encapsulé dans une protection (timeout + capture d'erreur) et possède un repli. Le pire scénario n'est jamais une erreur 500, mais un pack légèrement moins riche.

7.2 Cartographie des services

Service	Donnée fournie	Repli
Tavily	Recherche web (vols, hôtels, événements)	Données simulées
PredictHQ	Événements réels (concerts, festivals)	Tavily + LLM
Foursquare	Restaurants réels géolocalisés	Yelp
Yelp	Restaurants (repli de Foursquare)	Liste vide
OpenWeatherMap	Météo temps réel	Météo simulée par l'IA
Unsplash	Photo de destination	Pexels → image par défaut
Pexels	Photo (repli d'Unsplash)	Image par défaut
LLM (Gemini...)	Assemblage du pack	Cascade puis mocks

Tableau 7.1 — Services externes et leur stratégie de repli.

7.3 Les chaînes de repli en détail

Trois chaînes de repli illustrent le principe :

- **Restaurants** : Foursquare est interrogé en premier (1000 requêtes/jour gratuites). Si la réponse est vide, Yelp prend le relais. Si les deux échouent, un `.catch(() => [])` renvoie une liste vide — le pack est généré sans restaurants, jamais en erreur ;
- **Événements** : PredictHQ d'abord (procédure en deux temps : résolution de l'identifiant de lieu, puis recherche d'événements). En cas d'échec, repli sur une recherche Tavily interprétée par le LLM ;
- **Photos** : Unsplash, puis Pexels, puis une image par défaut.

Dérive de documentation détectée

L'en-tête du fichier `photo.ts` mentionne une source Wikipedia (« Wikipedia → Unsplash → Pexels ») qui **n'est pas implémentée** dans le code : la chaîne réelle est Unsplash → Pexels → défaut. Ce type d'écart entre commentaire et code est un rappel qu'une documentation doit refléter le code réel — précisément l'objet de ce rapport.

7.4 Protection des clés d'API

Aucune clé d'API n'apparaît jamais côté client. Les photos, par exemple, transitent par un **proxy backend** : le frontend appelle `/api/photos/:city`, et c'est le serveur qui interroge Unsplash avec sa clé secrète.

La clé reste confinée aux variables d'environnement du serveur, invisible dans le bundle JavaScript livré au navigateur.

7.5 Timeouts et robustesse

Chaque appel réseau externe est borné dans le temps par un `AbortController` (par exemple 15 s pour Tavily, 8 s pour Foursquare). Un service lent ne peut pas bloquer indéfiniment la génération. Cette discipline du timeout, combinée à la capture systématique des erreurs, garantit que le pipeline se termine toujours dans un délai maîtrisé.

8 · L'algorithme de scoring

8.1 Pourquoi un scoring déterministe ?

Une fois le pack assemblé, il est noté par un **algorithme déterministe** — c'est-à-dire sans intervention d'IA. Ce choix est délibéré et important : pour des entrées identiques, le score est toujours identique. Cette propriété rend le score **reproductible, explicable et testable**, là où un score produit par un LLM serait variable et impossible à valider par des tests unitaires.

Séparation IA / déterministe

Le système combine intelligemment deux natures de calcul : l'IA pour la **créativité** (assembler un pack pertinent et rédigé), un algorithme classique pour l'**évaluation** (noter ce pack de façon stable). On confie à chaque approche ce qu'elle fait le mieux.

8.2 Pondération par mode de voyage

Le score est une somme pondérée de critères, et la **pondération dépend du mode**. Un voyage « luxe » et un voyage « étudiant » ne valorisent pas les mêmes choses : le premier privilégie la qualité de l'hôtel, le second le prix.

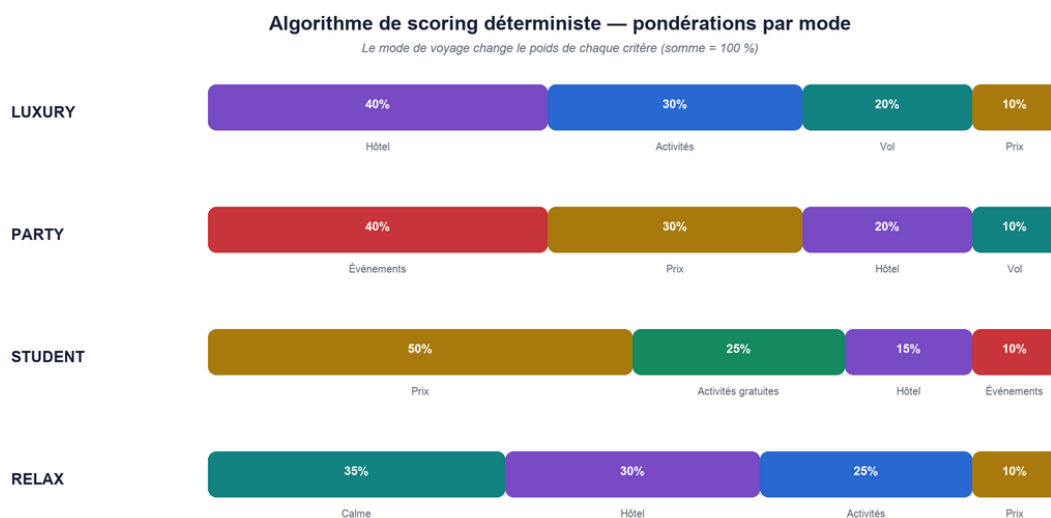


Figure 8.1 — Pondérations du score selon le mode de voyage : chaque mode redistribue l'importance des critères.

```
const MODE_WEIGHTS = {  
  luxury: { hotel: 0.40, activities: 0.30, vol: 0.20, prix: 0.10 },  
  party: { events: 0.40, prix: 0.30, hotel: 0.20, vol: 0.10 },  
  student: { prix: 0.50, activities_free: 0.25, hotel: 0.15, events: 0.10 },  
  group: { hotel: 0.35, activities: 0.30, prix: 0.20, vol: 0.15 },  
  relax: { calme: 0.35, hotel: 0.30, activities: 0.25, prix: 0.10 },  
};
```

8.3 Exemple : le mode « luxe »

En mode luxe, l'hôtel pèse 40 % du score (un cinq-étoiles maximise ce critère), les activités 30 % (une activité haut de gamme est valorisée), tandis que le prix ne pèse que 10 % — le budget n'est pas le facteur limitant. La même pack noté en mode « étudiant » obtiendrait un score très différent, car le prix y compte pour 50 %.

8.4 Résultat structuré

Le scoring renvoie un score global entre 0 et 1, accompagné du détail par critère — ce qui permet à l'interface d'expliquer la note à l'utilisateur :

```
{
  "total": 0.78,
  "details": {
    "vol": 0.82, "hotel": 0.90, "events": 0.30,
    "activities": 0.75, "prix": 0.50
  }
}
```

9 - Le modèle de données

9.1 Vue d'ensemble

La base PostgreSQL compte **six tables** articulées autour de l'utilisateur. Le schéma entité-association ci-dessous met en évidence les relations un-à-plusieurs et les suppressions en cascade qui garantissent la cohérence référentielle.

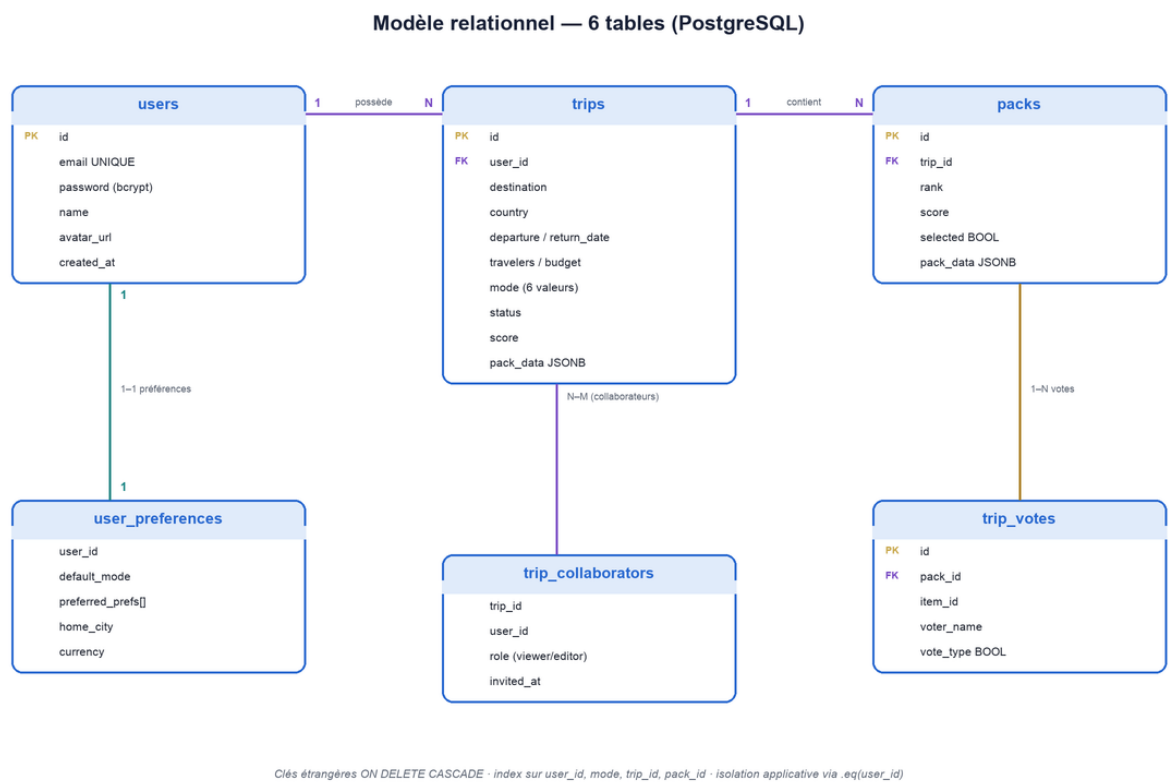


Figure 9.1 — Schéma entité-association : utilisateurs, voyages, packs, votes, préférences et collaborateurs.

9.2 Les tables principales

Table	Rôle	Relation
users	Comptes (email unique, mot de passe haché)	1 utilisateur → N voyages
trips	Voyages (destination, mode, pack JSONB, statut)	Appartient à un utilisateur
packs	Packs générés / variantes d'un voyage	1 voyage → N packs
trip_votes	Votes consensus par élément de pack	Cible un pack précis (pack_id)
preferences	Préférences utilisateur (relation 1-1)	1 utilisateur ↔ 1 préférence
collaborators	Collaborateurs invités sur un voyage	N utilisateurs ↔ N voyages

Tableau 9.1 — Les six tables et leurs relations.

La table `trips` illustre le modèle hybride : des colonnes typées pour les champs interrogeables (destination, mode, statut, dates) et une colonne `pack_data` de type **JSONB** pour stocker le pack complet, objet semi-structuré et volumineux.

9.3 Choix de conception

- **Clés primaires UUID** (`gen_random_uuid()`) plutôt que des entiers auto-incrémentés : les identifiants ne sont pas devinables ni énumérables, ce qui réduit la surface d'attaque par énumération ;
- **Suppression en cascade** (`ON DELETE CASCADE`) : supprimer un utilisateur supprime automatiquement ses voyages, packs et votes — pas de données orphelines ;
- **JSONB pour le pack** : souplesse documentaire pour une charge utile dont la forme évolue, tout en restant dans une base relationnelle ;
- **Statut de voyage** contraint à un ensemble de valeurs (`draft`, `confirmed`, `archived`).

9.4 Isolation des données par utilisateur

L'isolation des données entre utilisateurs est assurée **au niveau applicatif** : chaque requête de lecture ou d'écriture filtre systématiquement par `user_id`. C'est la barrière qui empêche un utilisateur A de consulter les voyages d'un utilisateur B.

```
// Chaque requete filtre par l'utilisateur authentifie
supabase.from('trips')
  .select('*')
  .eq('user_id', req.user.id);
```

Pourquoi pas le RLS de Supabase ?

Le Row Level Security de PostgreSQL/Supabase ne s'active qu'avec Supabase Auth. TripGenie utilise une authentification custom (JWT signé par le serveur) et la **clé de service** Supabase, qui contourne le RLS. La sécurité repose donc entièrement sur le filtrage applicatif. C'est un compromis assumé et documenté ; à grande échelle, je migrerais vers Supabase Auth + RLS pour une **défense en profondeur** (deux barrières plutôt qu'une).

9.5 Protection contre l'injection SQL

Le client Supabase construit des **requêtes paramétrées** : les valeurs fournies par l'utilisateur ne sont jamais concaténées dans une chaîne SQL, mais transmises comme paramètres distincts. Une tentative d'injection (`' ; DROP TABLE users; --`) est traitée comme une simple valeur de chaîne, pas comme du code SQL. C'est la défense de référence contre l'injection, ici obtenue sans écrire de SQL brut.

10 · Authentification et sécurité

La sécurité est traitée ici comme une **défense en profondeur** : plusieurs barrières indépendantes, de sorte que la défaillance d'une seule ne compromet pas le système. Ce chapitre couvre l'authentification, le stockage des secrets, et la cartographie des menaces.

10.1 Authentification par JWT en cookie httpOnly

L'authentification est **stateless** : aucune session n'est stockée côté serveur. À la connexion, le serveur vérifie le mot de passe, signe un jeton JWT contenant l'identité de l'utilisateur, et le dépose dans un **cookie httpOnly**.

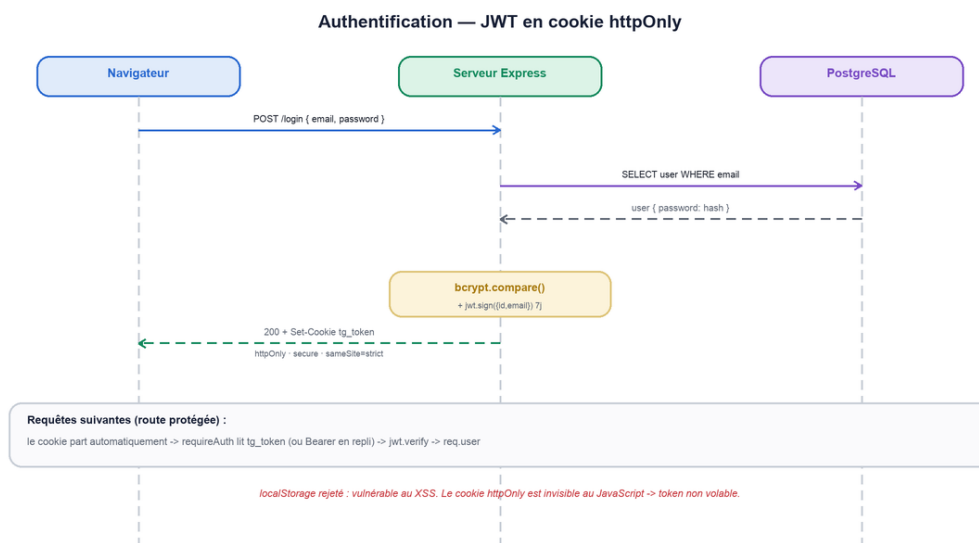


Figure 10.1 — Séquence d'authentification : connexion, dépôt du cookie httpOnly, requêtes authentifiées ultérieures.

```
res.cookie('tg_token', token, {
  httpOnly: true, // inaccessible au JavaScript du navigateur
  secure: true, // HTTPS uniquement (en production)
  sameSite: 'strict', // protection CSRF
  maxAge: 7 * 24 * 60 * 60 * 1000 // 7 jours
});
```

Aux requêtes suivantes, le navigateur renvoie automatiquement le cookie. Le middleware `requireAuth` extrait le jeton — d'abord du cookie, avec un repli sur l'en-tête `Authorization: Bearer` — le vérifie, et attache l'utilisateur décodé à `req.user`.

10.2 Pourquoi un cookie httpOnly plutôt que localStorage ?

C'est une question de sécurité classique et discriminante :

- **localStorage** est accessible en JavaScript. Une faille XSS (injection de script) permet alors de lire le jeton et de le voler ;
- **Cookie httpOnly** est inaccessible au JavaScript par conception. Même en cas d'XSS, le script ne peut pas lire le jeton.

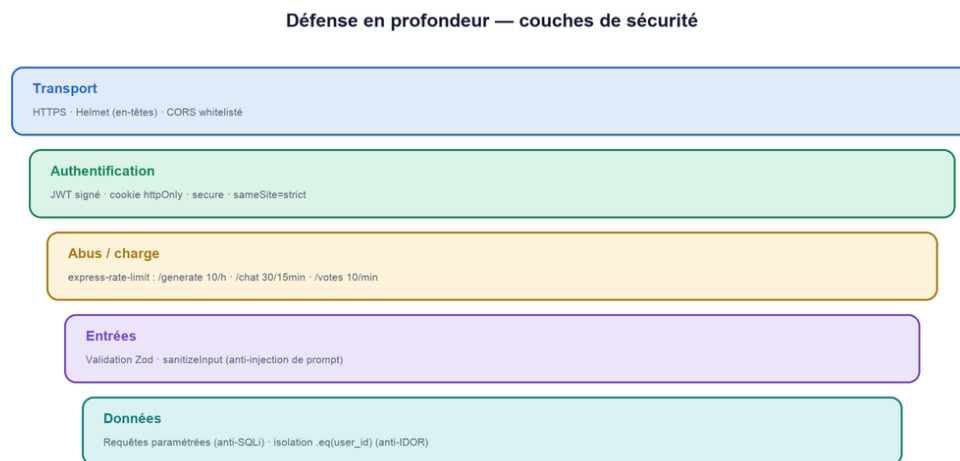
Choix de sécurité fort

Le cookie `httpOnly` + `sameSite:strict` neutralise deux familles d'attaques d'un coup : le **vol de jeton par XSS** (le script ne peut pas lire le cookie) et le **CSRF** (le navigateur n'envoie pas le cookie sur une requête initiée par un site tiers). C'est ce que permet le déploiement `same-origin` (chapitre 3).

10.3 Hachage des mots de passe

Les mots de passe ne sont **jamais stockés en clair**. Ils sont hachés avec **bcryptjs**, un algorithme volontairement lent et à facteur de coût configurable. Cette lenteur, contre-intuitive, est une protection : elle rend les attaques par force brute et par dictionnaire économiquement irréalistes. Le sel intégré à `bcrypt` empêche par ailleurs les attaques par tables précalculées (*rainbow tables*). À la connexion, `bcrypt.compare()` compare le mot de passe fourni au haché stocké, sans jamais le déchiffrer.

10.4 Défense en profondeur — cartographie



Cœur protégé : les données utilisateur. Chaque couche traverse une menace différente (XSS, CSRF, DDoS, injection, IDOR).

Figure 10.2 — Les couches de défense empilées, du réseau jusqu'à la donnée.

Menace	Contre-mesure
Vol de jeton JWT	Cookie <code>httpOnly</code> + <code>sameSite strict</code>
XSS (injection de script)	Jeton inaccessible au JS + en-têtes Helmet
CSRF	<code>sameSite strict</code> sur le cookie
Injection SQL	Requêtes paramétrées (client Supabase)
Entrées malveillantes	Validation Zod sur les routes sensibles
Force brute / DDoS	Limiteurs de débit sur les routes coûteuses
Exposition de clé d'API	Proxy backend + variables d'environnement
Accès inter-utilisateurs (IDOR)	Filtrage <code>.eq('user_id', ...)</code> systématique
Mot de passe compromis	Hachage <code>bcrypt</code> lent + sel

Tableau 10.1 — Menaces couvertes et contre-mesures associées.

10.5 Attaques explicitement testées

Une suite de tests de sécurité dédiée (`auth-tokens.test.ts`) valide la résistance à des attaques concrètes — ce qui couvre directement les concepts de la spécialisation cybersécurité :

- **Jeton expiré** → rejeté (401) ;
- **Signature invalide** → rejeté (401) ;
- **Attaque `alg:none`** : un jeton forgé sans signature (algorithme « none ») est rejeté — le serveur impose l'algorithme de vérification ;
- **IDOR** (*Insecure Direct Object Reference*) : l'utilisateur A ne peut pas accéder aux voyages de l'utilisateur B → 404, pas 403, pour ne pas révéler l'existence de la ressource ;
- **Provenance du jeton** : cookie contre en-tête Bearer, les deux chemins testés.

10.6 Limites de sécurité connues

Dette de sécurité assumée

Trois limites sont identifiées et inscrites à la feuille de route : (1) **pas de jeton de rafraîchissement** — une session dure 7 jours puis expire brutalement, sans rotation ; (2) **pas de limiteur de débit sur `/api/auth/login`**, ce qui laisse la porte au bourrage d'identifiants ; (3) l'usage de la **clé de service Supabase**, très privilégiée, concentre le risque — une compromission du serveur donne un accès total à la base. La parade serait un retour à Supabase Auth + RLS.

11 · Le frontend React

11.1 Structure SPA et routage

Le frontend est une **Single Page Application** : un unique fichier HTML est chargé, puis React Router gère toute la navigation côté client sans rechargement de page. Le composant racine `App.tsx` déclare cinq routes et installe les fournisseurs globaux (client React Query, routeur, système de notifications).

Route	Page	Rôle
/	Home	Onboarding + génération de pack
/trips	Trips	Liste des voyages sauvegardés
/trip/:id	TripDetail	Détail + chat de modification
/login	Login	Authentification
/preferences	Preferences	Réglages utilisateur

Tableau 11.1 — Routes de l'application React.

11.2 Les maquettes des écrans

Les maquettes ci-dessous présentent les écrans principaux et leur agencement, du desktop au mobile.



Figure 11.1 — Maquettes des écrans principaux : accueil/onboarding, résultats du pack, liste des voyages, et vue mobile.

11.3 Gestion d'état

Comme détaillé au chapitre 4, l'état est séparé en deux : **Zustand** pour l'état client (quatre stores : recherche, chat, authentification, thème), **React Query** pour l'état serveur (cache des voyages, votes, préférences avec `staleTime` de 5 minutes et une tentative de `retry`). Chaque store Zustand persiste sélectivement dans le `localStorage` via `partialize` — sans jamais y stocker de secret.

11.4 Communication avec l'API

Toutes les requêtes HTTP passent par une fonction `request()` centralisée. Le point essentiel est l'option `credentials: 'include'`, qui ordonne au navigateur d'**envoyer le cookie httpOnly** à chaque appel — sans quoi l'authentification par cookie ne fonctionnerait pas.

```
async function request(path, opts = {}) {
  const res = await fetch(`${BASE}${path}`, {
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include', // envoie le cookie httpOnly
    ...opts
  });
  const data = await res.json();
  if (!res.ok) throw new Error(data.error || `Erreur ${res.status}`);
  return data;
}
```

11.5 Expérience utilisateur

- **Sonner** — notifications *toast* non bloquantes pour le retour d'action ;
- **Framer Motion** — transitions et animations fluides et déclaratives ;
- **Recharts** — visualisation du budget ventilé en graphique ;
- **Leaflet** — carte interactive localisant les activités du pack.

11.6 Responsive et accessibilité

L'interface a été adaptée au mobile : le chat de modification, autrefois masqué sur petit écran, est désormais un *bottom sheet* animé déclenché par un bouton flottant. L'accessibilité clavier a été ajoutée sur les cartes cliquables (`role`, `tabIndex`, gestion des touches Entrée et Espace, anneau de focus visible). L'export PDF du pack utilise l'impression native du navigateur (`window.print()`).

12 · Conteneurisation Docker

12.1 Pourquoi Docker ?

Docker encapsule l'application et ses dépendances dans une **image** reproductible. L'intérêt est triple : éliminer le syndrome du « ça marche sur ma machine » (l'environnement est figé), faciliter la portabilité vers n'importe quel hébergeur compatible conteneurs, et rapprocher l'environnement de développement de celui de production.

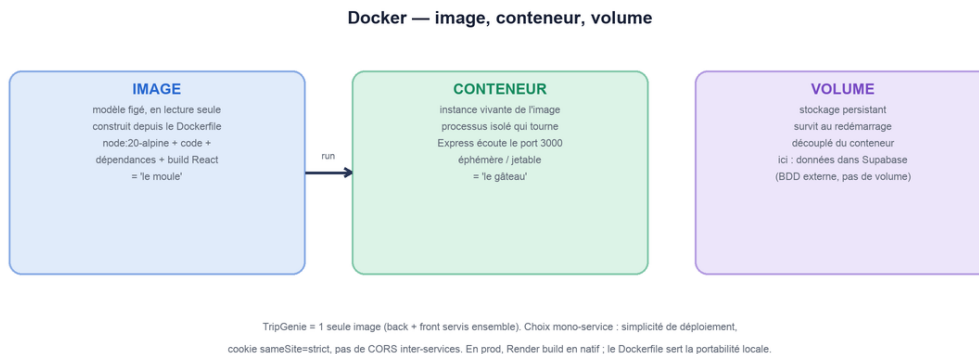


Figure 12.1 — Concepts Docker : du Dockerfile à l'image, puis au conteneur en exécution.

12.2 Dockerfile multi-étapes

Le Dockerfile est **multi-étapes** : une étape de build compile le TypeScript serveur et le bundle React, une étape finale ne conserve que les artefacts de production et les dépendances nécessaires. Cette séparation produit une image finale plus légère, sans les outils de compilation.

```
# Etape 1 – build
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build          # compile serveur (tsc) + client (vite)

# Etape 2 – runtime
FROM node:20-alpine
WORKDIR /app
COPY --from=build /app/dist-server ./dist-server
COPY --from=build /app/client-react/dist ./client-react/dist
COPY package*.json ./
RUN npm ci --omit=dev
EXPOSE 3000
CMD ["npm", "start"]
```

12.3 Orchestration locale (docker-compose)

Un fichier `docker-compose.yml` orchestre l'application en local, simplifiant le démarrage (un seul `docker compose up`) et la gestion des variables d'environnement.

Docker en production ?

En production, le déploiement n'utilise **pas** Docker mais le *runtime* Node natif de Render (voir chapitre 13). Le Dockerfile existe pour la **portabilité** et la reproductibilité locale : il garantit qu'on peut migrer vers une plateforme de conteneurs (Kubernetes, ECS...) sans réécriture si le besoin se présente. C'est un choix pragmatique — ne pas payer la complexité Docker en production tant que le PaaS suffit, tout en gardant la porte ouverte.

13 · Déploiement en production

13.1 Hébergement sur Render (PaaS)

L'application est déployée sur **Render**, une plateforme PaaS (*Platform as a Service*). Render prend en charge l'infrastructure (serveur, HTTPS, certificats, redémarrage automatique) ; il ne reste qu'à décrire le service. Le déploiement utilise le *runtime* Node natif (`runtime: node`) : Render exécute directement les commandes de build et de démarrage, sans conteneur Docker.

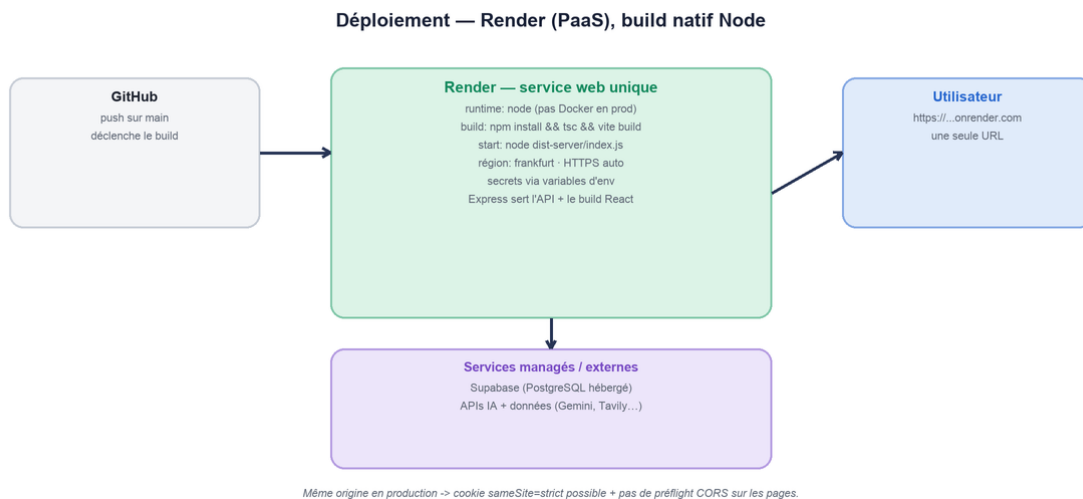


Figure 13.1 — Architecture de déploiement : du dépôt Git au service unique Render servant l'API et la SPA.

13.2 Configuration déclarative (render.yaml)

Le déploiement est décrit de façon **déclarative** dans `render.yaml` — une approche *infrastructure as code* : la configuration est versionnée avec le code, reproductible et auditable.

```
services:
  - type: web
    name: tripgenie-api
    runtime: node
    buildCommand: npm install && npm run build
    startCommand: npm start
    envVars:
      - key: NODE_ENV
        value: production
      - key: ALLOWED_ORIGINS
        value: https://tripgenie.onrender.com
```

13.3 Build et démarrage

La commande de build enchaîne la compilation du serveur TypeScript (`tsc`, sortie dans `dist-server`) et le build du client (Vite, sortie dans `client-react/dist`). Au démarrage, le serveur Express sert l'API sous `/api` et les fichiers statiques du build React — concrétisant le déploiement same-origin évoqué au chapitre 3.

13.4 Variables d'environnement

Toutes les valeurs sensibles (secret JWT, clés d'API, identifiants Supabase) sont injectées via des **variables d'environnement**, jamais committées dans le dépôt. Le détail des variables figure en annexe E.

Dérive détectée dans `.env.example`

Le fichier `.env.example` est partiellement désynchronisé du code : il omet plusieurs clés réellement lues (Gemini, Foursquare, Yelp, PredictHQ, OpenWeather, Pexels) et conserve une clé obsolète (Eventbrite). Mettre à jour ce fichier modèle est une correction simple mais importante pour qu'un nouveau développeur configure correctement le projet.

14 · Scalabilité

14.1 État actuel : une instance unique

Dans sa configuration actuelle, TripGenie tourne sur une **instance unique** Render. C'est un choix proportionné à l'usage visé (démonstration, portfolio, trafic modéré). Il serait malhonnête de prétendre que l'application est prête pour des millions d'utilisateurs ; en revanche, plusieurs décisions d'architecture **n'hypothèquent pas** la montée en charge future.

Un **load balancer** (répartiteur de charge) n'a de sens qu'à partir du moment où l'on exploite *plusieurs* instances : son rôle est d'aiguiller le trafic entre elles. Avec une instance unique, il n'y a rien à répartir — son absence est donc cohérente, pas un oubli. La figure ci-dessous illustre l'évolution possible.

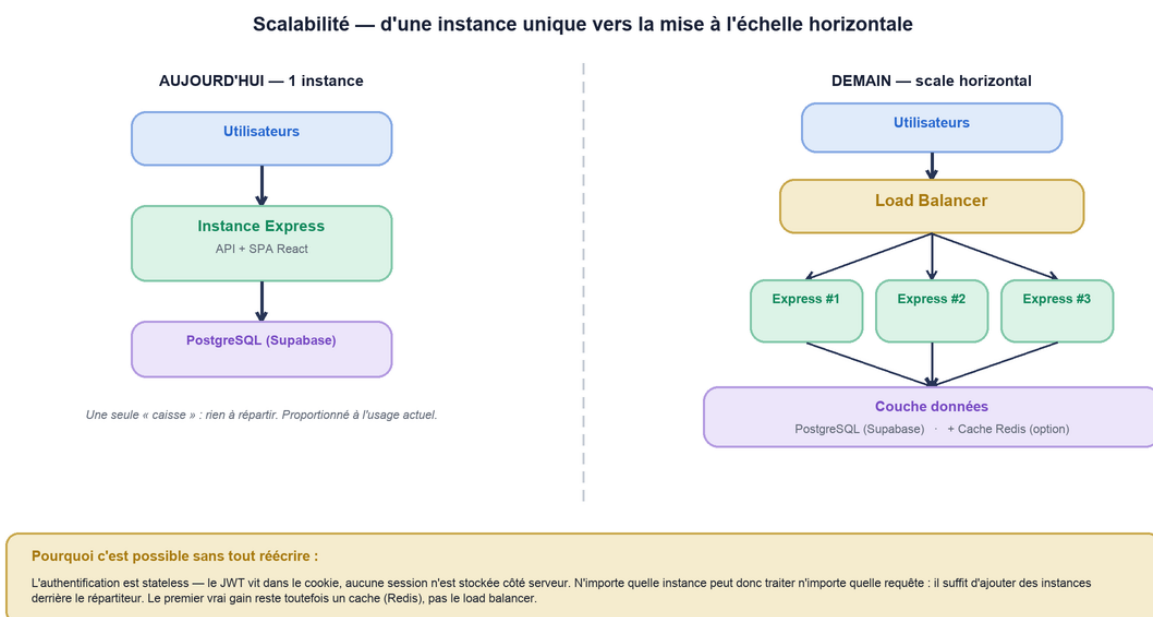


Figure 14.1 — D'une instance unique (aujourd'hui) à la mise à l'échelle horizontale derrière un load balancer (demain), rendue possible par l'authentification stateless.

14.2 Ce qui facilite la montée en charge

- **Authentification stateless** : aucune session n'est stockée côté serveur. On peut donc ajouter des instances derrière un répartiteur de charge sans avoir à partager un état de session — toute instance peut traiter n'importe quelle requête ;
- **Recherches parallélisées** (`Promise.allSettled`) : le temps de génération est borné par le service externe le plus lent, pas par leur somme ;
- **Cache côté client** (React Query) : les données déjà chargées ne sont pas re-demandées pendant 5 minutes, ce qui allège le serveur ;
- **PostgreSQL** : base éprouvée, capable de fortes volumétries avec index et pooling.

14.3 Goulots d'étranglement identifiés

- **Instance unique** : pas de mise à l'échelle horizontale ni de redondance — l'instance est un point de défaillance unique ;

- **Latence et coût des LLM** : chaque génération dépend d'appels externes lents et payants ; c'est le facteur de coût dominant ;
- **Quotas des API tierces** : Foursquare, PredictHQ, etc. imposent des limites quotidiennes ;
- **Absence de cache serveur** : deux utilisateurs demandant le même voyage déclenchent deux générations complètes.

14.4 Trajectoire d'évolution

Si le trafic l'exigeait, l'évolution se ferait par étapes, dans l'ordre du meilleur rapport bénéfice/effort :

1. **Mise à l'échelle horizontale** : plusieurs instances derrière un répartiteur — rendue possible par l'auth stateless ;
2. **Cache serveur (Redis)** : mémoriser les packs récents pour des destinations/modes identiques, économisant des appels LLM ;
3. **File d'attente** : traiter les générations en tâche de fond pour absorber les pics ;
4. **CDN** pour les fichiers statiques du frontend ;
5. **Pooling de connexions** PostgreSQL et migration vers Supabase Auth + RLS pour la défense en profondeur.

15 · Stratégie de tests

15.1 Organisation par couche

Le projet compte une suite d'environ **17 fichiers de tests** (Vitest + Supertest), organisée en miroir de l'architecture en couches. Cette structuration par responsabilité rend explicite ce qui est testé à chaque niveau.

Couche	Cible	Exemples
unit	Fonctions isolées, sans HTTP	Scoring par mode, recherche d'hôtel (timeout)
services	Services externes mockés	Foursquare, Yelp, PredictHQ
security	Authentification et entrées	Signup, login, jetons JWT, validation Zod
integration	Pipeline complet via HTTP	Génération avec restaurants fusionnés

Tableau 15.1 — Organisation des tests par couche.

15.2 Le principe des mocks

Tous les services externes sont **simulés** en test, pour des raisons de coût, de vitesse et de déterminisme — un test ne doit jamais dépendre d'un réseau ou d'une clé d'API :

- Les **LLM** sont remplacés par des fonctions contrôlées renvoyant un pack fixe ;
- **Supabase** est mocké par une chaîne d'appels factice (from → select → eq → single) ;
- **Foursquare / Yelp / PredictHQ** : `global.fetch` est remplacé avant l'import du service ;
- Les **limiteurs de débit** sont neutralisés en test — sans quoi la suite se bloquerait elle-même après quelques requêtes.

15.3 Un patron Vitest important : `vi.hoisted()`

Vitest *hisse* (hoist) les appels `vi.mock()` en haut du fichier, avant les imports. Les constantes déclarées plus bas sont donc inaccessibles dans la fabrique de mock, ce qui provoque une `ReferenceError`. La solution est `vi.hoisted()`, qui s'exécute, lui aussi, avant les imports :

```
// CORRECT – vi.hoisted() s'exécute avant les imports
const { mockSingle } = vi.hoisted(() => {
  const mockSingle = vi.fn();
  return { mockSingle };
});
vi.mock('supabase', () => ({ single: mockSingle })); // OK
```

Maîtriser ce détail démontre une compréhension fine du cycle d'exécution des tests, au-delà du simple usage de l'API.

15.4 Couverture

- **Chemins critiques** (*golden path*) de bout en bout ;
- **Sécurité** : `bcrypt`, `cookie httpOnly`, attaque `alg:none`, IDOR, expiration de jeton ;
- **Validation** : toutes les routes IA face à des entrées invalides ;

- **Chaînes de repli** : bascule Foursquare → Yelp, fusion des restaurants dans le pack.

16 · Points forts, points faibles et feuille de route

16.1 Points forts

- **Cohérence full-stack TypeScript** avec types partagés client/serveur — contrat de données unique ;
- **Résilience par conception** : `Promise.allSettled`, cascade multi-LLM, chaînes de repli — l'application ne tombe jamais entièrement ;
- **Sécurité réfléchie** : cookie `httpOnly` + `sameSite strict`, `bcrypt`, `Zod`, filtrage par `user_id`, attaques testées ;
- **Séparation IA / déterministe** : créativité au LLM, évaluation reproductible à l'algorithme de scoring ;
- **Honnêteté du cadrage** : « pipeline orchestré », pas « agent autonome » ;
- **Tests structurés par couche** avec mocks systématiques.

16.2 Points faibles

- **Vols au départ codés en dur** (Paris/CDG) au lieu de l'origine réelle de l'utilisateur — bug fonctionnel ;
- **Pas de limiteur de débit sur la connexion** — exposition au bourrage d'identifiants ;
- **Pas de jeton de rafraîchissement** — session de 7 jours sans rotation ;
- **Clé de service Supabase** très privilégiée + RLS contourné — risque concentré sur le serveur ;
- **Dérives de documentation** : `.env.example` incomplet, commentaire Wikipedia non implémenté dans `photo.ts`, mocks étiquetés Ibiza mais contenant Saint-Tropez ;
- **Incohérence de valeur par défaut** : ville d'origine « Bordeaux » dans un store, « Paris » ailleurs ;
- **Pas de cache serveur** des générations — coût et latence redondants.

16.3 Feuille de route priorisée

Priorité	Action	Bénéfice
Haute	Propager l'origine réelle jusqu'au générateur de vols	Corrige un bug visible
Haute	Limiteur de débit dédié sur <code>/api/auth/login</code>	Bloque le credential stuffing
Haute	Synchroniser <code>.env.example</code> avec le code	Onboarding développeur fiable
Moyenne	Jeton de rafraîchissement + rotation	Sessions plus sûres
Moyenne	Cache Redis des packs générés	Coût et latence réduits
Basse	Migration Supabase Auth + RLS	Défense en profondeur BDD
Basse	Mise à l'échelle horizontale + CDN	Tenue de charge

Tableau 16.1 — Feuille de route d'amélioration, par priorité.

17 · Conclusion

TripGenie démontre la maîtrise d'un cycle complet de développement web : conception d'une architecture trois tiers, choix technologiques justifiés, mise en œuvre d'un backend sécurisé, orchestration d'un pipeline d'intelligence artificielle résilient, modélisation relationnelle, frontend réactif, conteneurisation et déploiement automatisé.

Au-delà des fonctionnalités, le projet illustre une **posture d'ingénieur** : chaque décision est argumentée par rapport à ses alternatives, les compromis sont assumés et documentés, et les faiblesses sont identifiées plutôt que dissimulées. C'est précisément cette capacité à raisonner sur ses choix — et non la seule capacité à produire du code — qui distingue un développeur autonome.

Les axes d'amélioration listés au chapitre 16 ne sont pas des aveux d'échec mais une **feuille de route** : ils témoignent d'un regard critique sur son propre travail et d'une compréhension claire de ce qui sépare un projet pédagogique d'un produit à l'échelle industrielle. C'est, en définitive, l'objectif d'un titre RNCP 5 : former un professionnel capable de concevoir, réaliser et faire évoluer une application web de bout en bout.

Annexe A - Correspondance avec le référentiel RNCP 5 DWWM

Le titre RNCP 5 Développeur Web et Web Mobile s'articule autour de deux blocs de compétences. Le tableau suivant relie chaque compétence à sa concrétisation dans TripGenie.

Bloc 1 — Développer la partie front-end sécurisée

Compétence	Mise en œuvre dans TripGenie
Maquetter une application	Maquettes des écrans (fig. 11.1), parcours utilisateur défini (§1.4)
Réaliser une interface statique et adaptable	React + Tailwind, responsive (bottom sheet mobile), accessibilité clavier
Développer une interface dynamique	SPA React Router, Zustand + React Query, animations Framer Motion
Réaliser une interface accessible	role/tabIndex, navigation clavier, anneaux de focus

Tableau A.1 — Bloc 1 (front-end).

Bloc 2 — Développer la partie back-end sécurisée

Compétence	Mise en œuvre dans TripGenie
Mettre en place une base de données	PostgreSQL, 6 tables, UUID, clés étrangères en cascade (ch. 9)
Développer les composants d'accès aux données	Client Supabase, requêtes paramétrées, filtrage user_id
Développer la partie back-end	API REST Express, routes CRUD, validation Zod, pipeline IA
Élaborer et mettre en place un déploiement	render.yaml (IaC), Docker, build automatisé (ch. 12-13)
Sécuriser l'application	JWT httpOnly, bcrypt, Helmet, rate limiting, tests de sécurité (ch. 10)

Tableau A.2 — Bloc 2 (back-end).

Annexe B - Retour façon jury

Cette annexe restitue un retour critique tel qu'un jury RNCP ou un examinateur Holberton pourrait le formuler. Elle distingue les points forts à valoriser, les faiblesses à anticiper, et les questions les plus probables — accompagnées d'éléments de réponse.

B.1 Points forts à valoriser

Ce qui impressionne un jury

(1) La **résilience assumée** du pipeline (allSettled + cascade LLM + replis) montre une pensée « production », pas seulement « ça marche ». (2) Le **raisonnement comparatif** sur chaque techno (pourquoi pas Next, pourquoi pas Redux, pourquoi pas Mongo) prouve l'autonomie de décision. (3) La **sécurité testée** (alg:none, IDOR) dépasse le niveau attendu. (4) Le **cadrage honnête** « pipeline orchestré, pas agent autonome » inspire confiance.

B.2 Points faibles à anticiper

Ce qu'un jury va sonder

Le jury cherchera les failles. Les plus exposées : les **vols codés en dur** (Paris/CDG) — à reconnaître spontanément ; l'**absence de limiteur sur le login** ; la **clé de service Supabase + RLS contourné**. Le bon réflexe n'est pas de cacher ces points mais de **les présenter soi-même** avec leur correctif, comme au chapitre 16. Un candidat qui connaît ses faiblesses rassure davantage qu'un candidat qui les ignore.

B.3 Questions probables et éléments de réponse

« Votre application utilise-t-elle un agent IA autonome ? »

Non. C'est un **pipeline orchestré** : étapes fixes et ordonnées (valider → rechercher → assembler → noter). Seul le chat de modification est agentique, car le modèle y choisit dynamiquement quoi changer. Cette distinction est développée au chapitre 6.

« Pourquoi TypeScript et pas Python comme en formation ? »

Pour avoir **un seul langage** sur toute la chaîne et **partager les types** entre client et serveur. Le contrat de données est unique : si le serveur change la forme d'un objet, le client ne compile plus. Python/Flask aurait imposé un second écosystème et une duplication des modèles.

« Pourquoi allSettled plutôt que all ? »

all échoue si une seule promesse échoue : une panne météo arrêterait toute la génération. allSettled attend toutes les promesses ; le pack est produit avec les données disponibles. C'est le fondement de la dégradation gracieuse.

« Comment sécurisez-vous la session utilisateur ? »

JWT signé, déposé dans un cookie **httpOnly + sameSite strict + secure**. httpOnly neutralise le vol par XSS (le JS ne lit pas le cookie), sameSite strict neutralise le CSRF. Le mot de passe est haché avec bcrypt. Limite connue : pas encore de jeton de rafraîchissement.

« Votre RLS Supabase est-il activé ? »

Non, et c'est un compromis documenté. Le RLS suppose Supabase Auth ; or j'utilise une auth custom et la clé de service qui le contourne. La sécurité est applicative (`.eq('user_id', ...)`). À l'échelle, je migrerais vers Supabase Auth + RLS pour deux barrières au lieu d'une.

« Comment votre application monte-t-elle en charge ? »

Aujourd'hui : instance unique, suffisante pour l'usage visé. L'auth **stateless** permet d'ajouter des instances derrière un répartiteur sans état partagé. Les étapes suivantes seraient un cache Redis des packs et une file d'attente pour les générations (chapitre 14).

« Qu'est-ce qu'un code HTTP 409 ? »

Un **conflit** avec l'état actuel de la ressource : typiquement un email déjà enregistré. À distinguer du 400 (requête mal formée) — ici la donnée est valide, c'est son unicité qui pose problème.

« Si vous aviez plus de temps, que corrigeriez-vous en premier ? »

Les trois priorités hautes du chapitre 16 : propager l'origine réelle aux vols (bug visible), ajouter un limiteur sur le login (sécurité), et resynchroniser `.env.example` (fiabilité de l'onboarding développeur).

Annexe C · Glossaire

Terme	Définition
SPA	Single Page Application — une seule page HTML, navigation gérée côté client sans rechargement
JWT	JSON Web Token — jeton signé portant l'identité, permettant une auth sans session serveur
httpOnly	Attribut de cookie le rendant inaccessible au JavaScript (protection anti-XSS)
CSRF	Cross-Site Request Forgery — attaque exploitant l'envoi automatique des cookies
XSS	Cross-Site Scripting — injection de script malveillant dans une page
IDOR	Insecure Direct Object Reference — accès à une ressource d'autrui via son identifiant
RLS	Row Level Security — filtrage des lignes au niveau base de données
LLM	Large Language Model — grand modèle de langage (Gemini, Claude...)
JSONB	Type PostgreSQL stockant du JSON binaire indexable
PaaS	Platform as a Service — hébergement gérant l'infrastructure (Render)
IaC	Infrastructure as Code — infrastructure décrite et versionnée (render.yaml)
Dégradation gracieuse	Capacité à fonctionner en mode réduit malgré une panne partielle

Tableau C.1 — Glossaire des termes techniques.

Annexe D · Référence des routes API

Méthode	Route	Description
POST	/api/auth/signup	Création de compte
POST	/api/auth/login	Connexion
POST	/api/auth/logout	Déconnexion
GET	/api/auth/me	Profil courant
POST	/api/ai/generate	Génération de pack
POST	/api/ai/chat	Modification conversationnelle
POST	/api/ai/onboarding	Onboarding conversationnel
POST	/api/ai/destinations	Suggestion de destinations
GET	/api/trips	Liste des voyages
GET	/api/trips/:id	Détail d'un voyage
GET	/api/trips/share/:id	Voyage partagé (public)
DELETE	/api/trips/:id	Suppression d'un voyage
POST	/api/votes	Vote sur un élément
GET	/api/votes/:pack_id	Votes d'un pack
GET / PUT	/api/preferences	Préférences utilisateur
GET	/api/photos/:city	Proxy photo (clé masquée)
GET	/api/health	Sonde de santé

Tableau D.1 — Référence complète des routes.

Annexe E · Variables d'environnement

Les variables ci-dessous sont injectées par l'hébergeur et ne sont jamais committées. Celles marquées d'un astérisque (*) sont lues par le code mais absentes ou obsolètes dans `.env.example` — à resynchroniser (cf. §13.4).

Variable	Rôle
PORT / NODE_ENV	Port d'écoute / environnement
JWT_SECRET / JWT_EXPIRES_IN	Secret de signature / durée de validité du jeton
SUPABASE_URL	URL du projet Supabase
SUPABASE_SERVICE_KEY	Clé de service (contourne le RLS)
GEMINI_API_KEY *	LLM principal
ANTHROPIC_API_KEY	LLM secondaire (Claude)
OPENROUTER_API_KEY	LLM de repli
TAVILY_API_KEY	Recherche web
FOURSQUARE_API_KEY *	Restaurants
YELP_API_KEY *	Restaurants (repli)
PREDICTHQ_API_KEY *	Événements
OPENWEATHER_API_KEY *	Météo
UNSPLASH_ACCESS_KEY / PEXELS_API_KEY *	Photos
ALLOWED_ORIGINS / CLIENT_URL	Origines CORS autorisées

Tableau E.1 — Variables d'environnement (* = à resynchroniser dans `.env.example`).

Annexe F · Liens avec la formation Holberton

Module Holberton	Application dans TripGenie
SQL (web_back_end)	Tables PostgreSQL, UUID, clés étrangères en cascade, requêtes filtrées
ES6 Promises	Promise.allSettled pour les recherches parallèles non bloquantes
RESTful API	Routes Express, verbes et codes HTTP corrects (200/201/400/401/404/429)
Authentication	JWT + bcrypt — principes des modules Basic/Session authentication
Python OOP (HBnB)	Séparation des responsabilités routes → services → données
HBnB Part 4	JS + JWT cookie + Fetch transposés en React + cookie httpOnly
TDD / unittest	Tests Vitest + Supertest : mocks, golden path, cas limites
Sécurité web	alg:none, IDOR, XSS via cookie httpOnly — concepts cybersécurité
APIs REST externes	Foursquare, Yelp, PredictHQ — intégration et chaînes de repli

Tableau F.1 — Transposition des acquis Holberton dans le projet.

